

Problem Set 1

Applied Stats II

Due: February 11, 2026

Instructions

- Please show your work! You may lose points by simply writing in the answer. If the problem requires you to execute commands in `R`, please include the code you used to get your answers. Please also include the `.R` file that contains your code. If you are not sure if work needs to be shown for a particular problem, please ask.
- Your homework should be submitted electronically on GitHub in `.pdf` form.
- This problem set is due before 23:59 on Wednesday February 11, 2026. No late assignments will be accepted.

Question 1

The Kolmogorov-Smirnov test uses cumulative distribution statistics test the similarity of the empirical distribution of some observed data and a specified PDF, and serves as a goodness of fit test. The test statistic is created by:

$$D = \max_{i=1:n} \left\{ \frac{i}{n} - F_{(i)}, F_{(i)} - \frac{i-1}{n} \right\}$$

where F is the theoretical cumulative distribution of the distribution being tested and $F_{(i)}$ is the i th ordered value. Intuitively, the statistic takes the largest absolute difference between the two distribution functions across all x values. Large values indicate dissimilarity and the rejection of the hypothesis that the empirical distribution matches the queried theoretical distribution. The p-value is calculated from the Kolmogorov- Smirnov CDF:

$$p(D \leq d) = \frac{\sqrt{2\pi}}{d} \sum_{k=1}^{\infty} e^{-(2k-1)^2\pi^2/(8d^2)}$$

which generally requires approximation methods (see Marsaglia, Tsang, and Wang 2003). This so-called non-parametric test (this label comes from the fact that the distribution of the test statistic does not depend on the distribution of the data being tested) performs

poorly in small samples, but works well in a simulation environment. Write an R function that implements this test where the reference distribution is normal. Using R generate 1,000 Cauchy random variables (`rcauchy(1000, location = 0, scale = 1)`) and perform the test (remember, use the same seed, something like `set.seed(123)`, whenever you're generating your own data).

As a hint, you can create the empirical distribution and theoretical CDF using this code:

```
1 # create empirical distribution of observed data
2 ECDF <- ecdf(data)
3 empiricalCDF <- ECDF(data)
4 # generate test statistic
5 D <- max(abs(empiricalCDF - pnorm(data)))
```

Answer 1

Firstly, empirical data is created with 1000 Cauchy random variables.

```
1 data_PS1 <- rcauchy(1000, location = 0, scale = 1)
```

The implemented procedure corresponds to a simplified version of the Kolmogorov–Smirnov goodness-of-fit test, which uses cumulative distribution statistics to assess the similarity between the empirical distribution of observed data and a specified probability density function. A function is created, `D_statistic`. It takes as input the empirical data and returns as output the D-statistic, the p-value and an evaluation of the test. The function initially computes the empirical distribution of the previously created data; afterwards, the maximum absolute difference between the empirical distribution and a normal distribution. This step clarifies that we are testing whether the observed empirical distribution could be equal to a normal distribution.

A `for` loop is then created to compute the denominator of the p-value formula, looping over each observation. Once this quantity is obtained, the p-value is calculated..

The purpose of this function is to test whether the empirical distribution is equal to a normal distribution; thus, the null hypothesis supports this statement. A p-value smaller than 0.05 would indicate that the two distributions, empirical and normal, are statistically different.

```
1 D_statistic <- function(data){
2
3   ECDF <- ecdf(data)
4
5   empiricalCDF <- ECDF(data)
6
```

```

7  D <- max(abs(empiricalCDF - pnorm(data)))
8
9  denominator <- 0
10
11  for (i in seq_along(empiricalCDF)) {
12
13    denominator <- denominator +
14      exp(-((2 * i - 1)^2) * pi^2 / (8 * D^2))
15  }
16
17  p_value <- (sqrt(2*pi) / D) * denominator
18
19  results_test <- if (p_value < 0.05) {
20    paste("The distributions are statistically different,", p_value, "smaller
21      than 0.05.",
22      "The null hypothesis is rejected.")
23  } else {
24    paste("The distributions are not statistically different,", p_value, "bigger
25      than 0.05.",
26      "The null hypothesis is accepted.")
27  }
28
29  return(list(D = D, p_value = p_value,
30             results_test = results_test))

```

The function is finally run using as input the Cauchy data initially created. The results follow, comprising the D-statistic, the associated p-value and an evaluation of the null hypothesis with respect to a significance level of 0.05.

```

1 result_PS1 <- D_statistic(data_PS1)
2
3 print(result_PS1)

```

\$D

[1] 0.1347281

\$p_value

[1] 5.652523e-29

\$results_test

[1] "The distributions are statistically different, 5.65252281681864e-29
smaller than 0.05. The null hypothesis is rejected."

Question 2

Estimate an OLS regression in R that uses the Newton-Raphson algorithm (specifically BFGS, which is a quasi-Newton method), and show that you get the equivalent results to using `lm`. Use the code below to create your data.

```
1 set.seed (123)
2 data_PS1_2 <- data.frame(x = runif(200, 1, 10))
3 data_PS1_2$y <- 0 + 2.75*data_PS1_2$x + rnorm(200, 0, 1.5)
```

Answer 2

Initially, the function to be minimized is defined. This function corresponds to the OLS Residual Sum of Squares. It takes as arguments a vector of parameters `betas`, as well as the variables `y` and `x`. Inside the function, the estimated values `y_hat` are computed as β_0 (the first element of the vector `betas`) plus the product of β_1 (the second element of the vector `betas`) and `x`. The residuals are defined as the difference between the observed values of `y` and the estimated values.

To estimate the parameters, the function is optimized using the BFGS method. The initial values for both parameters are set equal to zero.

```
1 min_RSS <- function(betas, y, x) {
2   beta0 <- betas[1]
3   beta1 <- betas[2]
4   y_hat <- beta0 + beta1 * x
5   residuals <- y - y_hat
6   sum(residuals^2)
7 }
8
9 estimated_betas <- optim(par = c(0,0),
10   fn = min_RSS,
11   y = data_PS1_2$y,
12   x = data_PS1_2$x, ,
13   method = "BFGS")
```

Afterwards, `model_lm` is estimated using the `lm` function in R.

```
1 model_lm <- lm(y ~ x, data = data_PS1_2)
```

The estimated coefficients coincide up to numerical precision, confirming the equivalence between the two approaches.

Table 1: Coefficient estimations - BFGS optimisation vs LM

	lm	BFGS
(Intercept)	0.139	0.139
x	2.727	2.727